

导演台7.0 接入 FaceRefine 修脸功能 — 实战经验总结

日期： 2026-09-24 ~ 09-26 (**v1.2 补充版**：第 1-8 章为 09-24 初稿；第 9 章为上线后迭代复盘；第 10 章为 09-26 官方节点签名改序事故复盘) | **对象：** ComfyUI-H3-Director (导演台7.0, H3DirectorStudio) | **结果：** 已实现、首测通过、经一次崩溃事故完成三层修复，并经一次上游签名变更事故完成 4 处调用点加固

1. 任务与最终规格

给导演台 7.0 增加一个**可选** `face_refine` 输入入口，行为对齐 MiniMaxH3Director:

场景	行为
不接线	完全不修脸，零执行影响，老工作流/老缓存零变化
接线 + 二采关	一采解码后 修脸 (= 最终成片帧)
接线 + 二采开	二采解码后 修脸 (一采的中间帧不碰；二采失败回退一采时修回退帧)

实现规模: `studio_node.py` 15 处修改 (272,423 → 277,471 字节) + 新增 `face_refine/` 移植包 6 个文件。
`py_compile` 全量通过 + 独立冒烟测试通过。

2. 关键认知：两个插件的关系

- `ComfyUI_MiniMaxH3_Director` 与 `ComfyUI-H3-Director (7.0)` 是两个独立插件，主节点分别是 `MiniMaxH3Director` 和 `H3DirectorStudio`，互不相通。
- FaceRefine 节点是纯参数打包器**：没有视频输入、没有模型输入、不产出画面。它的唯一出口类型 `MMX_DIR_FACE_REFINE` 只是全局共享的类型字符串。
- 跨插件复用的正确姿势**：7.0 在自己节点上注册同名类型输入入口，即可直接接 MiniMax 插件的现成 FaceRefine 节点——两边共享的只有类型字符串和字典字段约定，Python 代码零耦合。
- 修脸的真正逻辑在宿主节点的 execute 里** (检测→裁剪→重采→贴回)，这决定了"给 7.0 加功能 = 移植运行时 + 在 7.0 的执行链里挂钩点"。

3. 架构分析方法：先摸清，再动手

动手前按固定清单摸底，本次全部命中：

- 主节点 INPUT_TYPES / RETURN / 执行入口**： `H3DirectorStudio` 自持 `model/clip/vae/audio_vae`，内部完成采样+解码——与 `MiniMaxH3Director` 同构，具备挂修脸的全部素材。
- 执行路径有几条**：发现**两条**——进程内 `direct() → _run_segment`，以及单段缓存重抽展开的外部图 `H3DirectorOfficialConditionCache + H3DirectorOfficialSampleCommit`。二者**最终都汇聚** `_sample_prepared_segment` → 确立唯一挂点，一改覆盖两路。
- 二采在哪一层工作**： `_second_sample_prepared_latent` 全程 **latent 域** (3D 放大 → 二次采样 → 最后才解码)，**不读一采解码后的像素** → 像素级修脸只能挂在"最终解码之后" (否则要多付一次整段 VAE 编解码回注)。

4. **模型预移位**: 7.0 图内自带 `MiniMaxH3SigmaShift`, 进节点的模型**已移位** → 修脸采样绝不能二次 `apply shift` (MiniMax 版 runtime 默认 `apply_shift=True`, 照抄会错)。
5. **缓存机制**: 段缓存键 = `_config_hash(run_cfg)` + 段 MP4 检查点; 条件缓存 (Qwen 编码) 与采样载荷分离。
6. `widgets_values` **按位置序列化** (历史经验 #38): 新增**自定义类型连线口不占 `widgets_values` 槽位** → 旧 workflow 加载零风险; 若要新增 widget 则必须追加在末尾。

4. 设计讨论: 挂点之争如何收束

讨论过三个方案:

- A. **最终成片帧** (MiniMax 的做法: "after Refine if wired")
- B. **一采后、二采前** (符合"修脸归生成"的直觉, 但二采开时需整段 decode + re-encode 回注 latent)
- C. **用户初案: "接入一采"**

收束关键是一条事实核查: 解析用户自己的工作流 JSON, 发现 `second_sample` 根本没配置 (= 二采关闭) → 在其实际用法下"接入一采"与"接入最终帧"**完全等价**, 争论失去意义。最终规格取二者之并集: **按二采开关自动选择挂点** (关=一采后, 开=二采后), 实现上仍是单一挂点 ("最终解码后"天然覆盖两种情况)。

经验: 架构之争先查用户的实际数据 (开没开二采), 比空对空推演各自优劣更快出结论。

5. 实施七条硬经验

1. **备份先行 (硬性习惯)**: 动手前 `studio_node.py` + 相关文件 (`__init__.py`、`nodes.py`, 本次未改也备) 全部复制为 `<原名>.bak_20260924_facerefine`, 并向用户报告路径。回退 = 备份覆盖 + 删除新增目录。
2. **缓存指纹的兼容策略**: 修脸参数**只在接线时**进 `run_cfg` 哈希 (`if face_pack: run_cfg["face_refine"] = fingerprint`) → 未接线时哈希逐位不变, **老段缓存全部保持命中**; 严禁 bump `CACHE_SCHEMA` (会导致全体用户缓存失效重跑)。
3. **两条执行路径都要覆盖**: 缓存重抽路径 (`SampleCommit`) 没有 `clip`, 而修脸条件编码需要 → 给它加**可选 `clip` 口**, 展开图 `GraphBuilder` 自动接线 (条件缓存节点本来就连着 `clip`, 接线现成)。修脸参数随**采样载荷 `sample_json`** 下传, **不进条件缓存**——改修脸参数不会触发 Qwen 重编码, 只重跑采样段。
4. **移植包解耦**: `track/stitch/inject` 原样移植; 砍掉对源插件 `lib.image_prep`、`frame_align`、`refine_pack` 的一切 import, 17k+5 帧对齐、32 倍数画布、sigmas 解析全部本地实现。跨插件只共享类型字符串, 源插件删了也不崩。
5. **快速失败**: `direct()` 开头就校验 ultralytics + 检测权重并**预加载 YOLO**——依赖问题在排队后第一秒报错, 而不是整段生成完才炸; 预加载放在正常模式 (非 `inference_mode`) 构造, 避免推理张量当权重的隐患。
6. **挂点的时序细节**: 放在解码后、**写盘前**、且在 `_apply_continuity` (尾帧桥接) 之前 → 修脸结果仍被锚帧校正, 段接缝自动保持一致。帧格式转换: `uint8 numpy` → `float 0..1 torch` → 修脸 → `uint8 numpy`, 进出与 `_write_decoded` 同构。
7. **双层验证 + grep 复核 (两次立功的铁律)**: ① 全量 `py_compile`; ② 独立冒烟测试 (归一化/指纹/JSON 往返/模块导入/权重解析); ③ **用 grep 把每个改动点逐一回读**。第 ③ 步先后抓到两处**编辑工具报成功但**

批次漏发的传参——第一次是 `direct() → _run_segment` 漏 `face_pack=face_pack`（进程内路径修脸永久静默失效），第二次是 `_expand_cached_reroll` 调用点漏 `clear_vram_before_face`（有默认值兜底，**零报错零症状**）。两处都会以"功能装了但没生效"的形态潜伏，只有回读能抓到——见 9.1。

6. 踩坑记录

坑	处理
PowerShell <code>Get-Content / ConvertFrom-Json</code> 默认按 ANSI 读 UTF-8 workflow JSON，中文变乱码、偶发解析失败	一律 <code>-Encoding UTF8</code>
<code>pwsh -c</code> 传多行 Python 脚本，内层双引号被原生参数传递吃掉	写临时 <code>.py</code> 文件再执行，用完删除
报告区锚点 <code>old_string</code> 不匹配	实际缩进是 8 空格而非 12—— 锚点必须以读盘结果为准，不能凭印象写
edit 工具单次成功 ≠ 批次全送达	批量编辑后必须 <code>grep</code> 全部关键标记回读核对（见第 5 条第 7 点）
Console 中文输出乱码（GBK）	只看 exit code 与关键 ASCII 字段，不以肉眼乱码判失败

7. 文件清单 · 验证 · 回退

新增（`ComfyUI-H3-Director\face_refine\`）：`__init__.py`、`pack.py`（归一化/指纹/JSON往返/前置检查）、`track.py`（检测跟踪裁剪）、`stitch.py`（贴回）、`inject.py`（AV latent 视频流回注）、`runtime.py`（7.0 版运行时：官方 `ReferenceToVideo` 条件 + 镜像 `_first_sample_prepared_latent` 采样）。

修改：`studio_node.py` 15 处——可选口、`direct` 收参与前置检查、`run_cfg` 指纹、`_run_segment / _sample_prepared_segment` 传参、唯一挂点、`_apply_face_refine` 方法、展开图 `face_pack+clip`、`SampleCommit` 可选 `clip` 口与收参、两路报告行、`prepared["condition_prompt"]`。

备份（全链 8 个，按时间序）：

时间戳	文件	对应改动
20260924_facerefine	<code>studio_node.py / __init__.py / nodes.py</code>	修脸 v1 接入
20260924_facevram	<code>studio_node.py</code>	脸修前清理显存开关
20260925_canvas512	<code>director_face_refine.py / director\face_refine\pack.py</code> （MiniMax 插件）	画布默认 768→512
20260925_stitchfix	<code>face_refine\runtime.py / face_refine\stitch.py</code>	A+B 崩溃修复
20260926_r2vkwards	站点+副本 各 <code>studio_node.py / face_refine\runtime.py</code>	官方签名改序：4 处改全关键字传参（见第 10 章）

验证：`py_compile` 全量 ☒ | 冒烟测试（JSON 往返、17k+5 对齐、标签清洗、权重解析）☒ | **09-25 实测通过**：512 画布双段修脸（56/175、61/192 帧）31:02 完整跑通无断连。

回退：按需取对应时间戳的 `.bak` 覆盖回原文件 + 删除 `face_refine\` 目录（仅针对 v1 接入本身）。

8. v1 边界与后续候选

已落地（详见第 9 章）：~~脸修前清显存开关~~  (9.1)、~~贴回访问违例崩溃修复~~  (9.2–9.4, 含画布默认 512)。

仍然成立的边界：

- 修脸条件 = 段提示词清洗后（去 `<Picture/Video/Audio>` 标签），**不带参考素材块**（注入的裁剪 latent 已是外观来源，denoise 0.4 保结构）。
- 未检测到人脸 → **原样保留成片** + 报告注明，不中断生成。
- 段接缝 12 帧渐隐未移植（7.0 尾帧桥接已部分覆盖）。

剩余候选： `export_pre_face_refine` 「输出修脸前」对比口（注意追加在 RETURN 末尾）、修脸条件注入参考素材、僵尸槽位驱逐 `_evict_dead_loaded_models`（若日志出现 `Potential memory leak detected with model` 则需移植，成因与你的 Sage 补丁链相同）。

9. 上线后迭代复盘（2026-09-24 晚 ~ 09-25, v1.1）

本章为初稿定稿后追加：首测 → 崩溃事故 → 参数与结构两轮修复 → 复测通过的完整过程。

9.1 「脸修前清理显存」BOOLEAN 开关

按选定方案（显式开关，对齐 MiniMax `clear_vram_before_face_refine`）实现：

- **UI：**追加在 `required` 末尾（每段生成上限 之后）——沿用 #38 位置安全模式，旧 workflow `widgets_values` 缺该值按默认 **False** 执行，零错位。
- **实现：**挂点在 `_apply_face_refine` 之前调用现成的 `_cleanup_runtime_resources(deep=True, reason="脸修前清理显存（开关已开启）")`；进程内路径参数透传，缓存重抽路径随 `sample_json` 携带 `face_vram_cleanup` 标志。
- **报告：**run 开头 `VRAM：脸修前清理显存已开启（...）`；段报告修脸行带前缀 `修脸前已深度释放显存 |`。
- **新原则（本章最有复用价值的一条）：**只改运行期行为、不改输出像素的参数 → 不进 `run_cfg` 缓存哈希。拨开关不会让缓存段重跑（输出一致，无需重跑）；与之对照，修脸参数（denoise/画布等）改变画面 → 必须进哈希。同一条哈希线，两类参数两种待遇。
- **第二次抓漏发：**`_expand_cached_reroll` 调用点漏传 `clear_vram_before_face`——有默认值兜底，**零报错零症状**，纯靠 grep 回读抓到。至此第 5 条经验双案例在身（第一次 `face_pack` 漏传同样是静默失效）——“批量编辑后必 grep 回读”升级为铁律。

9.2 首测崩溃事故复盘（access violation → 断连）

现象：09-24 首测第二次 run 的段2——修脸采样 5/5 完成、VAE 解码完成、进入贴回 → `face_refine/stitch.py:98`（`patch_can = refined_crops[...].to(dev)`，把 32 帧 768² 裁剪块 ~226MB 从内存拷进显存）→ `Windows fatal exception: access violation` → ComfyUI 进程终止 → 前端断连。

定位方法与教训：

- 栈精确到行后先误判为 `grid_sample`（对行号的想当然），逐行读文件才确认 98 行是 `.to(dev)` 拷贝——异步 CUDA 栈指向“故障浮出的行”，不一定是肇事操作；

- access violation 属 C++/CUDA 层，Python 不可捕获——与 #22 记录的"主机内存分配失败表现为 c10 访问违例而非可捕获 OOM"同族。

证据链：

#	证据	指向
1	同尺寸（192 帧）同代码：首 run 成功、第二 run 段2 失败	非逻辑 bug
2	差异变量 = 同进程第二个任务（首 run 是冷进程）	内存碎片/状态累积
3	段1 结束日志已达 内存压力安全线；修脸前可用显存 5.96 GiB < reserve 6 GiB（8GB 卡 #22 公式）	资源贴边
4	贴回时 20GB 流式 UNET + 4.9GB VAE 全在场，而贴回是纯张量运算、 不需要任何模型	资源释放时机错误
5	主机侧同时压着 base/裁剪/解码/贴回副本数 GB + pinned 池 6.5 GB ≈ 15 GB+（16GB 机器）	显存内存双重贴边

定性：边界资源态，非代码逻辑 bug。修复方向 = 9.3（参数降需）+ 9.4（时序释放 + 分块降峰）。

9.3 画布默认 768 → 512

- 提问："是不是不需要 768² 这么大"——成立。
- 分析：画布的唯一使命是让脸在 H3 眼里足够大（H3 渲染小脸差，是修脸功能存在的原因）。下限 512：crop_factor 2.5 下 512 画布 = 脸 205px，对 H3 已足够；低于 512 = 把小脸交回 H3，修脸失去意义。用户片段是 480p（脸 100-150px、裁剪 250-375px），512 装得下——768 是在放大已经够用的东西。
- 收益（768²→512² = 面积 44%）：采样耗时 -56%（5.5 分钟 → 2.4 分钟级）、VAE 编解码 -56%、贴回单次拷贝 226→100 MB、裁剪张量 1.36→0.6 GB。
- 改动 6 处默认值：director_face_refine.py（INPUT default ×2 + pack() 签名 ×2）、pack.py（pack_face_refine 签名、钳制兜底、normalize 兜底）。auto_capped_768 模式名与其 tooltip 保留 768（自适应上限语义，不该动）。
- 教训：默认值不追溯已保存节点——widgets_values 里存的是 768，旧节点必须手改或重拖；默认值只影响以后新放的节点。（#38 位置序列化的姊妹条。）
- 更优替代：canvas_mode = auto_capped_768（512 起步、按最大裁剪自适应上涨、上限 768）——小脸省算力、大特写不亏分辨率。
- 实测：09-25 09:48 run——canvas 512、双段修脸（56/175、61/192 帧）、31:02 完整跑通无断连。

9.4 A+B 结构性修复（贴回前释放 + 分块降峰）

- 修复 A • 贴回前释放模型：runtime.py 在裁剪解码完成、进 stitch_faces 之前调 _cleanup_runtime_resources(deep=True, reason="贴回前释放模型（修脸链路）")（函数内懒加载导入 + 兜底直调 unload_all_models / cleanup_models / soft_empty_cache）。贴回不需要任何模型——让 UNET/VAE 提前退场，拷贝变空场作业。日志新增可辨识行。
- 修复 B • 分块预算修正：stitch.py 原公式只按源帧估每帧字节（864×480≈2.4 MB），但贴回真正的显存大头是画布尺寸的 patch/mask（768²≈6.6 MB）——改为 max(源, 画布) 计算，上限 32→16 帧、预算 1024→256 MB：单次拷贝 226→113 MB（512 画布约 50 MB）。

- 二者均不改输出像素 → 不进缓存指纹，已命中段缓存保持有效。
- 验证：`py_compile` + 7 项标记 `grep` 回读 + 位置序校验（释放@6117 < stitch@6330）。备份 `.bak_20260925_stitchfix` ×2。
- 方法论提炼——内存边界崩溃三层保险：① 参数降需（画布 768→512，需求砍半）② 时序释放（不需要的资源在不需要它的阶段提前退场）③ 分块降峰（单次分配预算按真实大头计算）。三层各自独立生效，叠加后崩溃点余量从"贴边"变"宽裕"。

9.5 日志诊断速查 (`user/comfyui.log`)

想知道	看哪行	含义
FaceRefine 接线了吗	修脸已启用: <code>face_yolov8m.pt ... canva</code> <code>s=...</code>	每 run 首段前出现一次
某段修脸执行了吗	段N修脸完成: <code>...s \ 检测 xx/192 帧 ...</code>	出现即执行 (含"未检出人脸"跳过备注)
修脸开关生效了吗	深度释放: 脸修前清理显存 (开关已开 启) ...	采样解码后、修脸前
修复 A 生效了吗	深度释放: 贴回前释放模型 (修脸链路) ...	修脸采样解码后、贴回前
什么都没有	无任何修脸行	未接线 = 规格内零影响 (09-25 10:22 run 实测验证)

10. 上游签名改序事故复盘 (2026-09-26, v1.2)

本章为第二次上线后追加：一次与显存、内存、GPU 全都无关的失败——官方节点签名悄悄改序，把位置传参的老写法直接打成 `TypeError`。

10.1 现象与"第一反应"纠偏

- 现象：点开始生成，段1/3 在 8.8 秒内即失败，日志尾行 `!!! Exception during processing !!! MiniMaxH3ReferenceToVideo.execute() got multiple values for argument 'ref_image_size'`；前一行只有自己的摘要 [H3导演台] 段1/3失败: ... + 已保留完成段: 无。
- 第一反应 (错)：怀疑显存/环境，重启 ComfyUI 再试——同样报错，一字不差。
- 纠偏依据：`got multiple values for argument` 是纯 Python 参数绑定错误，发生在函数体执行之前，与 CUDA、显存、驱动、内存毫无关系。先看错误形态，再决定要不要重启，能省一轮试错。（对照 9.2 的 `access violation`：那才是资源边界问题。）

10.2 根因：官方 `execute()` 签名改序

官方 `comfy_extras\nodes_minimax_h3.py` 把 `vae / audio_vae` 挪到了 `ref_image_size` 之后：

版本	签名
老 (7.0 接入时按它写的)	<code>(clip, vae, audio_vae, prompt, width, height, length, ...)</code>
新 (当前)	<code>(clip, prompt, width, height, length, ref_image_size="match", vae=None, audio_vae=None, ...)</code>

而 `studio_node.py` 仍按老顺序位置传 7 个实参、后面再跟 `ref_image_size=` 关键字，实际绑定变成：

位置实参	落到的形参
<code>clip</code>	<code>clip</code>
<code>vae</code>	<code>prompt</code>
<code>audio_vae</code>	<code>width</code>
<code>condition_prompt</code>	<code>height</code>
<code>condition_width</code>	<code>length</code>
<code>condition_height</code>	<code>ref_image_size</code> ← 被第 6 个位置实参占掉
<code>length</code>	<code>vae</code>
<code>ref_image_size=...</code> (关键字)	→ 与位置实参冲突，抛 <code>TypeError</code>

关键认知：即使不冲突，参数也已整体错位、语义全错。所以此处必须**改写调用方式**，不能靠"把实参挪个位置"糊过去。

10.3 定位方法（三条，可复用）

1. **读 traceback，别只看自己打的摘要行：** `段1/3失败: ...` 是 catch 后的摘要，信息量低；真正的帧在 `Traceback` 段，逐帧读到 `studio_node.py:3668 in _condition` 即锁定肇事行。
2. **拿肇事行与官方当前签名逐参数对照：** 列出"位置实参 → 实际形参"错位表（见 10.2），冲突点一眼可见，且能同时看出"就算不报错也是全错"。
3. **grep 时排除 `.bak_*`：** 同类调用点会被历史备份大量命中，按 `*.py` 过滤（备份文件名不以 `.py` 结尾），归档不改。

10.4 修复：改全关键字传参（共 4 处）

```
return MiniMaxH3ReferenceToVideo.execute(
    clip=clip, vae=vae, audio_vae=audio_vae,
    prompt=condition_prompt, width=condition_width, height=condition_height,
    length=length, ref_image_size=ref_image_size, ref_images=ref_images,
    ref_audios=ref_audios, ref_videos=ref_videos,
    ref_video_audios=ref_video_audios)
```

树	文件	行
站点 D:\导演台7.0\ComfyUI\ComfyUI\custom_nodes\ComfyUI-H3-Director\	studio_node.py	3672
站点 (同上目录)	face_refine\runtime.py	111
副本 D:\临时保存\ComfyUI-H3-Director\ComfyUI-H3-Director\	studio_node.py	3672
副本 (同上目录)	face_refine\runtime.py	111

- 备份统一 *.bak_20260926_r2vkwargs (同目录) , py_compile 四个文件全过，两棵树里已搜不到旧的位置传参写法。
- face_refine\runtime.py 那处更隐蔽：它要等跑到修脸阶段才会暴露——若只修报错的主流程那处，功能会"先跑通一段、再在修脸时炸"。
- 改 .py 后必须重启 ComfyUI 才加载新代码（与 9 章同）。

10.5 三条新经验

1. 调官方节点一律全关键字传参（新铁律）：官方签名不是稳定契约，位置传参对上游改序零防御。同族坑在 ComfyUI_MiniMaxH3_Director\nodes\conditioning.py 第 135–137 行早有注释（那次表现成 TypeError: ... for //: 'str' and 'int'）——同一个根因第二次咬人。全关键字是对签名改序免疫的最便宜保险。
2. "grep 复核"升级为"扫同类隐患"：第 5 条第 7 点的 grep 铁律，此前用于"回读自己刚改的地方"；这次第一次用于找出本次尚未报错、但同病的其他调用点——一搜 MiniMaxH3ReferenceToVideo.execute(就是 4 处，报错那处只是"先被执行到"。只修报错点，修脸阶段必然再炸一次。（铁律第三次立功。）
3. 两棵树必须同步改：站点（D:\导演台7.0\...）与开发副本（D:\临时保存\ComfyUI-H3-Director\...）是两份独立代码，只修站点的话，副本下次部署即复发。

10.6 排除项（避免误伤）

- ComfyUI_StarNodes\minimax\star_minimax_aio.py 里也有 clip, vae, audio_vae, prompt, ... 的顺序，看着可疑，但那是它自己的内部函数 _build_conditioning（注释写明是官方 execute 的 in-node replica），签名自治、不调官方节点 → 不动。
- MiniMaxH3ImageToVideo.execute 的签名仍是 (clip, vae, prompt, width, height, length, first_frame, last_frame)，位置传参仍然正确 → 同一文件里另一处长得很像的调用不要顺手"统一改"。
- D:\临时保存\ComfyUI-H3-Director.bak_20260925_githubsync\ 归档里仍是老写法 → 归档不改。

10.7 日志速查 (9.5 节续)

想知道	看哪行	含义
是不是又被上游签名咬了	<code>got multiple values for argument '...' / got an unexpected keyword argument</code>	代码与官方签名不匹配， 重启无用 ，去比对签名
失败是参数问题还是资源问题	<code>段N/3失败: ...</code> 出现在几秒内、且日志无显存/解码行	参数类/导入类错误，不是资源问题
失败后导演台怎么处置	已保留完成段：无；重新运行将从段1继续；失败后已深度释放	自愈行为，缓存未被污染，可直接重跑

v1.2 补充版 (2026-09-26) 。全部改动涉及文件均已按时间戳分层备份；改 .py 后需重启 ComfyUI 方可生效。